Big Data Processing
COSC 2637/2633
Assignment 3 – Query Log Analysis Using Spark Streaming

Assessment Type	– This assignment may be completed **individually** or in **groups of two or three** students. – Submit online via Canvas → Assignment 3. – Marks awarded for meeting requirements as closely as possible. – Clarifications/updates may be made via announcements or relevant discussion forums.
Due Date	Due at 23:59, 25/10
Marks	25

Overview

Write Spark programs which gives you a chance to apply the essential components you learned in this course to understand the complexity of Spark programming.

Learning Outcomes

The key course learning outcomes are:

- CLO 1 - Model and implement efficient big data solutions for various application areas using appropriately selected algorithms and data structures.
- CLO 2 - Analyse methods and algorithms, to compare and evaluate them with respect to time and space requirements and make appropriate design choices when solving real-world problems.
- CLO 4 - Explain the Big Data Fundamentals, including the evolution of Big Data, the characteristics of Big Data and the challenges introduced.
- CLO 5 - Apply non-relational databases, the techniques for storing and processing large volumes of structured and unstructured data, as well as streaming data.
- CLO 6 - Apply the novel architectures and platforms introduced for big data.

Group Assessment

You may complete this assignment either individually or in a group of two or three students. No other group sizes are permitted. Groups must consist of students who are enrolled at the same level (either all Undergraduate COSC2633 students or all Postgraduate COSC2637 students). All members are expected to contribute equally. Unless a contribution dispute is reported (and supported by the work log), all group members will receive the same grade. Individual work is allowed but not encouraged. Please note that no bonus marks will be awarded for completing the work individually.

Group Formation and Locking Policy

- Navigate to **People** in Canvas, then select the **Groups** tab. **Scroll past the A1 and A2 groups until you reach the A3 group section.** Choose an empty A3 group and join it together with your group members. Please do not use your A1 or A2 group for this assessment. You may keep the same team members, but you must join a A3 group.
- **You are not permitted to create new groups. You are not permitted to rename the groups.** This restriction is in place because student-created groups are not linked to the grading system, which means they cannot be identified by the system during marking.
- Canvas groups are intended only for students completing the assignment in groups of two or three. Students completing the assignment individually do not need to join a Canvas group.
- Groups will be locked 72 hours before the deadline. **Once groups are locked, students can no longer join, leave, or modify their groups. This ensures consistency in grading and prevents last-minute changes that could affect group accountability.**

You must finalise and verify your group membership before submission and only submit after groups are locked. **Please do not submit your assignment until the groups have been finalised and locked.**

Group Leadership

The first student to join a group on Canvas is assigned as the group leader by default. However, this role can be changed later within the group settings if needed. The group leader is responsible for ensuring that the group is correctly finalised before the deadline. This includes:

- Confirming all required files are prepared and correctly named.
- Verifying the correct members are listed in the group.
- Submitting the final work before the deadline.
- Keeping and submitting the group work log as part of the submission.

The group leader is responsible for coordinating communication and organising weekly meetings. The role does not provide additional authority over other group members, nor does it make the group leader responsible for completing the assignment on behalf of the group or for the actions of other members. All group members share equal responsibility for contributing to the assignment, reviewing the final submission, and complying with the course's academic integrity requirements.

Extension Policy for Group Assignments

Any extension requests for group assignments must be approved by the Course Coordinator. Group extensions will only be considered under exceptional circumstances (**if more than 50% of the group members are unable to contribute at the same time**) and require supporting documentation. Reasons such as being unable to cope with study load or taking on additional work shifts are not considered sufficient grounds for an extension.

Please note that extensions are granted only for **unexpected circumstances outside a student's control**, as outlined in RMIT policy. Regular commitments such as part-time work or general workload are considered foreseeable and therefore do not typically meet the criteria. Students are expected to plan their work and make every effort to meet the original deadline.

The following reasons are **generally not considered valid** for granting an extension, as they are foreseeable or within a student's control:

- **Academic workload & time management:** multiple assignments due in the same week; general study pressure or heavy workload; poor time management or starting late; and/or underestimating the difficulty of the assignment.
- **Employment commitments:** regular part-time or casual work shifts; busy work schedule or increased workload, and/or choosing to take additional shifts.
- **Travel & personal plan:** pre-booked travel, holidays, or social commitments; events arranged in advance; and/or planned relocation or return home arrangements.
- **Group-related issues:** difficulty coordinating with group members; miscommunication within the group; one member being unavailable or unresponsive; and/or late group formation.
- **Technical & submission issues (avoidable):** uploading incorrect files; last-minute technical problems due to late submission; Internet issues at the time of submission (if preventable); and/or failure to check submission requirements.
- **Minor or foreseeable circumstances:** feeling stressed or overwhelmed (without supporting evidence); general tiredness or lack of preparation, and/or known commitments that were planned in advance.

If you wish to apply for an **individual extension**, you must withdraw from your group (if applicable) before the assignment deadline. You will then be required to complete and submit the entire assignment individually. If your individual application is approved, alternative tasks will be given only to you — not to your former group members.

What to Do If a Team Member Isn't Contributing

- Try to resolve the issue first through communication within the group. Document communication attempts (e.g., emails, messages). Keep a simple log of who completed which parts of the work.
- If the situation doesn't improve, you may choose to leave the group, join other groups OR complete the assignment individually.

Please notify the tutor and the course coordinator early if you decide to do so. Adjustments due to non-contributing members will not be granted unless clear evidence is provided.

Assessment Details

Modern web applications, search engines, learning platforms, and e-commerce systems generate large volumes of query logs every second. These logs record what users search for, when they search, which device they use, and whether the query produces successful results. Analysing query logs in real time can help organisations understand user behaviour, detect trending topics, identify failed searches, monitor system usage, and improve search quality.

The query log files are provided in CSV format. Each record represents a single user query submitted to the system. The schema is:

`query_id,user_id,query_text,event_time,device,result_count,response_time_ms`

where:

- `query_id` is the unique identifier of the query.
- `user_id` identifies the user who submitted the query.
- `query_text` contains the original search query.
- `event_time` records when the query was submitted.
- `device` indicates the device type used, such as desktop, mobile, or tablet.
- `result_count` indicates how many results were returned.
- `response_time_ms` records the query response time in milliseconds.

For example:

```
q001,u001,How to use “Spark Streaming” with Scala?,2026-05-01T09:00:05Z,desktop,12,180
q002,u002,Pig Latin UDF!!!,2026-05-01T09:00:08Z,mobile,5,240
q003,u003,Hadoop MapReduce example??,2026-05-01T09:00:11Z,tablet,0,310
```

In this assignment, you are required to use **Spark Streaming** to process query log files arriving in HDFS and generate **real-time** analytical outputs (the batch interval is 3 seconds).

A **single** Scala object contains implementations for all three of the following tasks. Namely, all three tasks are defined within the same Scala object (in one `.scala` file), with data flowing from Task 1 to Task 3 and an output generated after each task.

Text Processing Requirement (3 marks)

Before performing the following tasks, each `query_text` must be cleaned and normalised:

1. Case folding: convert text to lowercase.
2. Remove non-alphanumeric characters.
3. Remove extra spaces.
4. Split the query into terms.
5. Remove terms shorter than 3 characters.
6. Remove common stop words: `the, and, for, with, how, what, why, can, you, use, using`
7. Reconstruct the cleaned query text.

For example:

```
Original Query: How to use “Spark Streaming” with Scala?
Cleaned Query: spark streaming scala
```

Note: Records with an empty cleaned query must be ignored.

Task 1 – Cleaned Query Frequency Analysis (4 marks)

For each non-empty RDD in the DStream, count how many times each `cleaned query` appears in the current batch. The output must be

`cleaned_query,current_batch_count`

The output must be sorted by `current_batch_count` in descending order, and then by `cleaned_query` in ascending alphabetical order. This analysis provides a snapshot of user search activity within the current streaming batch. By counting the frequency of each cleaned query, organisations can identify which topics are attracting the most attention at a particular point in time. (See **Appendix A** for a sample output.)

Only non-empty RDDs should generate output. Save the output on HDFS.

Task 2 – Slow Query Incident Monitoring (8 marks)

Slow queries can negatively affect user experience, reduce system responsiveness, and indicate potential performance bottlenecks in the underlying search infrastructure. By analysing slow-query incidents in real time, organisations can identify problematic search terms, monitor the severity of performance issues, and understand how many users are affected.

For each RDD of Dstream, identify slow queries and produce a summary for each `cleaned query` in the current batch. A query is considered **slow** if its `response_time_ms` is **greater than** a specified `threshold`. The `threshold` must be provided as a command-line argument when the Spark Streaming application is executed.

For each `cleaned query`, calculate the following values:

- `slow_query_count` indicates how frequently the slow query occurred in the current batch.
- `max_response_time_ms` highlights the worst response time observed for the query.
- `average_response_time_ms` provides an overall measure of the query's performance.
- `affected_users` indicates how many distinct users experienced the slow query and therefore reflects the impact of the issue.
- `severity_level` classifies the incident according to its seriousness, helping system administrators prioritise investigation and remediation efforts. The `severity_level` should be assigned as follows:

```
CRITICAL  if max_response_time_ms >= threshold * 2
WARNING   otherwise
```

The output must be:

`cleaned_query,slow_query_count,max_response_time_ms,average_response_time_ms,affected_users,severity_level`

The output should be sorted by:

`severity_level ASC, max_response_time_ms DESC, slow_query_count DESC, cleaned_query ASC`

This ordering helps system administrators identify and prioritise the most critical performance issues.

1. Queries with a `CRITICAL` severity level are displayed before those classified as `WARNING`, ensuring that the most serious incidents receive immediate attention.
2. Within the same severity level, queries with higher `max_response_time_ms` are ranked higher because they represent the most severe performance degradation.
3. If multiple queries have the same severity and maximum response time, queries with a higher `slow_query_count` are prioritised because they indicate more frequent occurrences of the problem.
4. Finally, `cleaned_query` is sorted alphabetically to provide a consistent and reproducible ordering when all other ranking criteria are identical.

This prioritisation approach is commonly used in operational monitoring systems because it allows engineers to focus on the most critical and impactful performance problems before investigating less severe incidents.

Only non-empty RDDs should generate output. Save the output on HDFS. (See **Appendix A** for a sample output.)

Task 3 – Advanced Stateful Trending Query Analysis (10 marks)

Using `updateStateByKey`, maintain **accumulated** query statistics across all batches. For each cleaned query, maintain:

```
current_batch_count
previous_accumulated_count
accumulated_count
count_change
trend_status
```

where `count_change = accumulated_count - previous_accumulated_count`

The `trend_status` should be:

```
NEW
INCREASED
UNCHANGED
```

A query is `NEW` if it appears for the first time, `INCREASED` if its accumulated count increases in the current batch, and `UNCHANGED` if it already exists but does not increase in the current batch.
For each valid batch, output the top 10 most frequent queries based on accumulated statistics. The output should contain:

`cleaned_query,current_batch_count,previous_accumulated_count,accumulated_count,count_change,trend_status`

Ranking rules for the **top 10** trending queries:

1. Higher `accumulated_count` ranks higher.
   `accumulated_count` measures the total number of times a query has appeared since the streaming application started. Queries with higher accumulated counts represent long-term trends and therefore receive higher priority.
2. If tied, higher `current_batch_count` ranks higher.
   `current_batch_count` is used when multiple queries have the same accumulated count. This favours queries that are currently experiencing higher activity and helps identify emerging trends among queries with similar historical popularity.
3. If still tied, sort by `cleaned_query` in ascending alphabetical order.
   `cleaned_query` in ascending alphabetical order, provides a deterministic ordering when all numerical measures are identical. This ensures consistent and reproducible output across executions.

Together, the ranking provides a balanced view of both historical popularity and recent user activity, enabling organisations to identify sustained trends, emerging interests, and short-lived search patterns within the query stream. This approach is commonly used in streaming analytics systems where both historical popularity and current activity are important indicators of user interest. (See **Appendix A** for a sample output.)

Note: you are required to use `updateStateByKey` to continuously accumulative query statistics as new query log arrives. If checkpointing is enable, configure the checkpoint directory to `/sxxxxxxx/checkpoint` (replace `sxxxxxxx` with your student number). **This ensures each job uses its own unique checkpoint path.**

Undergraduate Requirement

At each 3-second interval, output the accumulated query statistics, regardless of whether new query log files have arrived in the monitored HDFS directory.

Postgraduate Requirement

More advanced task: only output the accumulated query statistics when the monitored HDFS directory receives new query log files. Namely, if no new query log files arrive during an interval, produce no output.

How We Will Run Your Work

To launch your job on YARN in client mode, execute:

```
spark-submit --class streaming.a3 --master yarn --deploy-mode client a3.jar hdfs:///input hdfs:///output 250
```

- `--class streaming.a3`: the name of the main class within your application's JAR file (`a3.jar`) that contains the main method to be executed must be `streaming.a3`.
- `a3.jar`: your assembled JAR file containing your compiled Scala code. **Ensure the JAR contains only the required libraries; exclude all irrelevant or unused dependencies.**
- `hdfs:///input`: the HDFS directory that the streaming job continuously monitors for new files.
- `hdfs:///output`: the target directory on HDFS for all output files produced by your application.
- `250`: slow-query threshold of **250 milliseconds**

Note: you must follow these instructions exactly; this demonstrates your ability to use `spark-submit` correctly as part of the assessment. No additional arguments beyond those specified may be passed to `spark-submit`.

Important Notes

Spark Streaming is designed for real-time or near-real-time data processing. You should test your program by adding new query log files into the monitored HDFS input directory while the streaming job is running.

Correct output alone is not sufficient. Your implementation should demonstrate a sound understanding of Spark Streaming concepts, including DStreams, RDD transformations, stateful processing, checkpointing, and output actions.

Functional Requirements

Thoroughly test your application in the specified EMR environment to avoid penalties.
Failure to follow the requirements incurs up to a **10-mark penalty**.

(a) For each task, the output on HDFS must be named with **a unique sequence number as a suffix**: `task1-001`, `task1-002`, `task2-001`, `task2-002`, `task3-001`, `task3-002` (**do not use other values such as the system timestamps as output folder names**).
Note: Follow the naming conventions precisely. The marking script will only recognize files and paths defined in the spec.

(b) For each task, if an RDD is empty, do not output. (An empty RDD in Spark is simply an RDD that contains zero elements.)

(c) For each task, the batch interval is 3 seconds.

(d) If checkpointing is enable, configure the checkpoint directory to `/sxxxxxxx/checkpoint` (replace sxxxxxxx with your student number).

(e) You need to create a single Scala project including all three tasks so that they work on the same stream data. Submitting more than one Scala file will result in automatic failure of the assignment.

(f) Pass the monitored and output directories as **spark-submit** arguments; do **not** hard-code them in your Scala code.

Submission Instructions

Assignments received late and without prior extension approval or special consideration will be penalised by a deduction of 10% of the total score possible per calendar day late for that assessment.

Your assignment should follow the requirements below and be submitted via Canvas > Assignment 3.
Assessment declaration: when you submit work electronically, you agree to the assessment declaration.

Failure to follow the format requirements incurs up to 5 marks penalty.

1. Prepare your zip file

- Place all required files into a single folder, including your JAR file (`a3.jar`) and scala file (*.scala) ONLY.
- **Do not include the IntelliJ project folder.**
- No subfolders.
- Compress the folder into a zip file.

2. Prepare a pdf file for Turnitin plagiarism check

- Copy and paste the **scala code** into a plain text editor.
- Save this as a **pdf file**.
- **Submit the pdf file separately from the zip file**. This pdf must be uploaded as a separate file to ensure it is properly processed by Turnitin for plagiarism checking.

3. Prepare your worklog doc file

- Please double-check that you have completed all two sections.
- Use the **Submission Declaration Checklist** to verify that your submission satisfies all submission requirements before submitting.

Name your zip file and pdf file using the following format:

`<GroupLeaderStudentID>_Group<GroupNumber>_<UG/PG>.zip`
`<GroupLeaderStudentID>_Group<GroupNumber>_<UG/PG>.pdf`
`<GroupLeaderStudentID>_Group<GroupNumber>_<UG/PG>.doc`

Replace:

- `GroupLeaderStudentID` with the student number of the group leader
- `GroupNumber` with your group number (e.g., 3)
- `PG` if you are in a postgraduate program; `UG` if you are in an undergraduate program

Example: `s1234567_Group3_UG.zip` and `s1234567_Group3_UG.pdf`
The group leader is responsible for submitting the group's final zip file. Duplicate submissions from the same group will not be accepted.

If you are completing A1 individually, name your zip file using the following format:

`<StudentID>_<UG/PG>.zip`
`<StudentID>_<UG/PG>.pdf`
`<StudentID>_<UG/PG>.doc`

Replace:

- `StudentID` with your actual student number
- `PG` if you are in a postgraduate program; `UG` if you are in an undergraduate program

Example**:** `s1234567_UG.zip` and `s1234567_UG.pdf`

Both group submissions and individual submissions are required to include a work log. For group submissions, the work log must include a member contribution section (Section 1) that clearly describes each member's contributions. For individual submissions, a work log is still required; however, the member contribution section is not applicable and does not need to be completed.

Important

- You must submit 3 files: **the zip file, the doc file, and the pdf file** for Turnitin plagiarism check.
- **Failure to submit the required pdf or doc file separately from the zip file will result in a 10% penalty (2.5 marks).**
- Submissions that do not include the required PDF or Word document will incur a 10% penalty for each missing file (2.5 marks per file).
- Submissions where the PDF **fails the Turnitin plagiarism check**, **will not be graded**.

- Please verify your submission carefully before the deadline. If you submit the **wrong file**, a **corrupted zip file**, or **a submission that does not comply with the required file structure**, any resubmission made after the deadline will be subject to the standard late submission penalty.
- Late submission penalties are applied automatically by the system based on the assessment due date and the time of submission. **The penalty is determined by when you submit your work, not when you complete it.** Therefore, even if you completed the assessment before the due date, a late submission would still incur the applicable late penalty.
- Work must run on the **RMIT-provided EMR** cluster.
- **Your submission must exclude the IntelliJ project folder — failure to do so carries a 10% penalty.**

Academic Integrity and Plagiarism (standard warning)

RMIT University treats plagiarism as a very serious offense constituting misconduct. Plagiarism covers a variety of inappropriate behaviours, including:

- Failure to properly document a source.
- Copyright material from the internet or databases.
- Collusion between students.

For further information on our policies and procedures, please refer to
https://www.rmit.edu.au/students/student-essentials/rights-and-responsibilities/academic-integrit

Appendix A – Sample input and output

Querylog_001.txt

```
q001,u001,How to use Spark Streaming with Scala?,2026-05-01T09:00:05Z,desktop,12,180
q002,u002,Pig Latin UDF!!!,2026-05-01T09:00:08Z,mobile,5,240
q003,u003,Hadoop MapReduce example??,2026-05-01T09:00:11Z,tablet,0,310
q004,u001,Spark Streaming Scala,2026-05-01T09:00:18Z,desktop,15,160
q005,u004,What is Spark Streaming?,2026-05-01T09:00:20Z,mobile,10,220
q006,u005,Apache Hive tutorial,2026-05-01T09:00:25Z,desktop,8,190
q013,u003,Hadoop MapReduce example,2026-05-01T09:00:29Z,mobile,2,520
q014,u012,Slow database query,2026-05-01T09:00:30Z,desktop,3,610
```

Querylog_002.txt

```
q007,u006,pig latin udf,2026-05-01T09:00:35Z,mobile,6,200
q008,u007,Spark streaming with Scala tutorial,2026-05-01T09:00:40Z,tablet,11,170
q009,u008,Apache Hive Tutorial!,2026-05-01T09:00:42Z,desktop,7,210
q010,u009,Kafka Streaming,2026-05-01T09:00:45Z,mobile,4,260
q011,u010,Hadoop MapReduce example,2026-05-01T09:00:50Z,desktop,0,350
q012,u011,How can you use Kafka streaming?,2026-05-01T09:00:55Z,mobile,5,280
q015,u010,Hadoop MapReduce example,2026-05-01T09:00:57Z,desktop,1,720
q016,u013,Slow database query,2026-05-01T09:00:58Z,tablet,2,490
q017,u014,Slow database query,2026-05-01T09:00:59Z,mobile,4,530
```

hadoop fs -cat /output/task1-001/*

```
hadoop mapreduce example,2
spark streaming scala,2
apache hive tutorial,1
pig latin udf,1
slow database query,1
spark streaming,1
```

hadoop fs -cat /output/task1-002/*

```
hadoop mapreduce example,2
kafka streaming,2
slow database query,2
apache hive tutorial,1
pig latin udf,1
spark streaming scala tutorial,1
```

hadoop fs -cat /output/task2-001/*

```
slow database query,1,610,610.00,1,CRITICAL
hadoop mapreduce example,2,520,415.00,1,CRITICAL
```

hadoop fs -cat /output/task2-002/*

```
hadoop mapreduce example,2,720,535.00,1,CRITICAL
slow database query,2,530,510.00,2,CRITICAL
kafka streaming,2,280,270.00,2,WARNING
```

hadoop fs -cat /output/task3-001/*

```
hadoop mapreduce example,2,0,2,2,NEW
spark streaming scala,2,0,2,2,NEW
apache hive tutorial,1,0,1,1,NEW
pig latin udf,1,0,1,1,NEW
slow database query,1,0,1,1,NEW
spark streaming,1,0,1,1,NEW
```

hadoop fs -cat /output/task3-002/*

```
hadoop mapreduce example,2,2,4,2,INCREASED
slow database query,2,1,3,2,INCREASED
kafka streaming,2,0,2,2,NEW
apache hive tutorial,1,1,2,1,INCREASED
pig latin udf,1,1,2,1,INCREASED
spark streaming scala,0,2,2,0,UNCHANGED
spark streaming scala tutorial,1,0,1,1,NEW
spark streaming,0,1,1,0,UNCHANGED
```